

# ControlPlane.ai — Admission-Control Layer for AI That Acts

Accenture Innovation Challenge 2026 · Round 2 · Team ControlPlane · PS #1

Public GitHub: <https://github.com/Srujan0798/controlplane-ai> · branch main · tag v0.2.0-round2

## 1. Portal uploads (five fields)

These are the only artifacts the submission portal needs. This PDF is field 3.

| # | Portal field            | Artifact                                                                                                  |
|---|-------------------------|-----------------------------------------------------------------------------------------------------------|
| 1 | Public GitHub link      | <a href="https://github.com/Srujan0798/controlplane-ai">https://github.com/Srujan0798/controlplane-ai</a> |
| 2 | Prototype video         | submission/ControlPlane_Round2_Prototype.mp4                                                              |
| 3 | README document (PDF)   | submission/ControlPlane_Round2_README.pdf                                                                 |
| 4 | Business proposal (PDF) | submission/ControlPlane_Round2_Proposal.pdf                                                               |
| 5 | Pitch deck (PPTX)       | submission/ControlPlane_Round2_Pitch.pptx                                                                 |

## 2. What this is

Every AI response that asks permission to **act** — issue a refund, send an email, publish a record — is a set of **claims**. ControlPlane captures provenance **outside** the model (STEP → SPAN → CLAIM → ACTION) and admits or holds each action through a frozen interlock matrix. Unproven claims cannot authorize irreversible actions. Lane 1 is deterministic; no LLM sits on the critical path.

This repository is the working prototype. The numbers in the evidence section are produced by `make eval` and `make bench` and are regenerated on every run — they are measured, not asserted.

### 3. The mechanism

Run any response text through extract → bind → entitle → interlock → shadow:

- **extract** — Lane-1 segmenter produces typed claims (numeric / structural / temporal / textual / derived) with hedging and role-in-action, no LLM.
- **bind** — each claim is matched to provenance spans: BM25 + lexical gate (textual), Indian/Western numeric normalization (numeric), symbol-table lookup (structural), recompute (derived). Middle band → UNKNOWN, never a default pass.
- **entitle** — span ACL is checked against principal clearance; an unbound PII entity is a leak (Rule A) → Block at R2/R3.
- **interlock** — a frozen 16-cell matrix maps (blast-tier × verdict-column) to one of five actuators. The matrix is never redrawn.
- **shadow** — every decision is dual-emitted; the published FNR carries a Wilson interval.

#### 4. Run it in 60 seconds (clean clone)

```
git clone https://github.com/Srujan0798/controlplane-ai && cd controlplane-ai  
python3 -m venv .venv && source .venv/bin/activate  
pip install -e ".[dev]"  
  
bash scripts/preflight-lite.sh # must PASS  
make run                       # http://127.0.0.1:8787  
  
# refund:   http://127.0.0.1:8787/?scenario=refund&mode=enforce&autorun=1  
# expected: show_text → Edit · issue_refund → Escalate (held)  
# gate:     http://127.0.0.1:8787/gate    (arbitrary text, no fixtures)  
make eval                                       # FNR with Wilson CI  
make verify                                   # tests + content laws + number freeze
```

Refund language is **held and escalated with the evidence packet** — never 'blocked'.

## 5. What to look at

- **/gate** — paste a response; watch claims, binding method + rationale, symbol table, matrix cell, actuator, evidence packet, dead compute, per-stage latency.
- **refund autorun** — show\_text → Edit · issue\_refund → Escalate (held).
- **examples/refund\_trace\_demo.py** — same dual-action with fixtures off; amount via numeric.
- **evals/cases/** — labelled cases incl. hard negatives; this is the corpus a judge can attack.
- **controlplane/interlock.py** MATRIX — the sixteen transcribed cells, sole decider.

## 6. Capability ledger

Every mechanism in the architecture, marked honestly: **implemented and tested** / **implemented, prototype-grade** / **designed, not built**. This preempts 'is any of this real?' by answering first.

| Mechanism                                 | Status                       | Where                         |
|-------------------------------------------|------------------------------|-------------------------------|
| Claim extraction (Lane 1, no LLM)         | implemented and tested       | controlplane/extract.py       |
| Numeric normalization (IN/US grouping)    | implemented and tested       | controlplane/numeric.py       |
| BM25 + lexical entailment gate            | implemented and tested       | controlplane/bm25.py          |
| Symbol table / structural lookup          | implemented and tested       | controlplane/symbols.py       |
| Binder v2 route dispatch                  | implemented and tested       | controlplane/binder.py        |
| PII Rule A (leak → Block)                 | implemented and tested       | controlplane/pii.py           |
| Entitlement / ACL audit                   | implemented and tested       | controlplane/entitlement.py   |
| Frozen 16-cell interlock matrix           | implemented and tested       | controlplane/interlock.py     |
| Lane deadlines (async gather)             | implemented and tested       | controlplane/lanes.py         |
| Hash-chained evidence ledger              | implemented and tested       | controlplane/ledger.py        |
| Override HTTP → chained ledger            | implemented and tested       | controlplane/server/app.py    |
| Analyze arbitrary text                    | implemented and tested       | POST /v1/controlplane/analyze |
| OpenAI-compatible FastAPI gate            | implemented and tested       | controlplane/server/app.py    |
| Dual-action refund (Edit + Escalate held) | implemented and tested       | examples/refund_trace_demo.py |
| Eval harness + Wilson CIs                 | implemented and tested       | evals/run.py                  |
| Load bench n=10000                        | implemented and tested       | scripts/load_bench.py         |
| published_fnr from last eval              | implemented and tested       | GET /v1/controlplane/metrics  |
| Shadow-replay threshold gate              | implemented and tested       | controlplane/feedback.py      |
| Canary auto-rollback                      | implemented and tested       | controlplane/feedback.py      |
| Jurisdiction packs (EU/IN/GDPR)           | implemented and tested       | policies/*.yaml               |
| Multi-turn claim inheritance              | implemented and tested       | controlplane/session.py       |
| Economist / dead-compute walk             | implemented and tested       | controlplane/economist.py     |
| Bias counterfactual (Lane 3)              | implemented, prototype-grade | controlplane/bias.py          |
| GPU / trained NLI adapter                 | designed, not built          | Lane-2 slot (stub)            |
| Production FNR on live traffic            | designed, not built          | shadow earn-out               |
| Production IAM repair                     | designed, not built          | out of Round 2 scope          |

## 7. Evidence (measured on this run)

Eval corpus: **168** labelled cases (426 bindings), **self-authored**. Ungrounded FNR **1.1%** (95% CI 0.2%–5.8%; n=94 ungrounded). We HOLD **98.9%** of ungrounded responses. Passable-action false-hold (FPR) **0.0%** (95% CI 0.0%–15.5%). Fail-closed caution on hard-negatives **64.2%**. Abstention (UNKNOWN rate) **0.5%** of bindings.

Published miss: struct-miss-000 (see evals/last\_run.json).

Per-route binding distribution:

| Route      | n   | supported | unknown | contradicted | unsupported | abstain |
|------------|-----|-----------|---------|--------------|-------------|---------|
| numeric    | 119 | 105       | 0       | 14           | 0           | 0.0%    |
| structural | 211 | 178       | 0       | 0            | 33          | 0.0%    |
| textual    | 71  | 50        | 2       | 0            | 19          | 2.8%    |

|          |    |    |   |    |   |      |
|----------|----|----|---|----|---|------|
| derived  | 12 | 2  | 0 | 10 | 0 | 0.0% |
| temporal | 11 | 11 | 0 | 0  | 0 | 0.0% |
| none     | 2  | 0  | 0 | 2  | 0 | 0.0% |

Load bench: n=10000 at concurrency sweep [1, 4, 16, 64]. Per-request compute p50=0.4332 ms, p95=0.5113 ms, p99=0.6129 ms, mean=0.4455 ms, max=8.4518 ms. Best throughput 2258.8 rps. (In-process compute, not API latency; 40ms is a Lane-1 target, never a measured p95.)

Refund language in this system is **held and escalated with the evidence packet** — never 'blocked'. Irreversible actions are not released without provenance.

## 7.1 Honest reading of these numbers

Measured on a **168-case self-authored corpus**. Ungrounded FNR **1.1%** (95% CI 0.2%–5.8%), passable FPR **0.0%** (95% upper bound 15.5%). Hard-negative hold rate is **64%** (64.2%, 95% Wilson CI 50.7%–75.7%; 53/168 = 31.5% hard negatives) — we over-flag, we measured it, and it is our named next milestone. No production traffic; production FNR is unknown. The corpus ships in the repo so you can attack it.

## 7.2 What we refuse to claim

This list is about **us**, not about anyone else. **We do not claim:** elimination of hallucinations · zero integration cost · zero added latency · one accuracy number · production-scale proof.

## 8. Repo map

controlplane/ — pipeline (extract, binder, symbols, bm25, numeric, pii, entitlement, interlock, lanes, feedback, bias, shadow, server)

evals/ — corpus (cases/), harness (run.py), Wilson CIs, last\_run.json

policies/ — jurisdiction packs (eu / in / gdpr / default)

scripts/ — bench, proposal/readme PDF builders, preflight, verify

examples/ — refund\_trace\_demo, knowledge\_flip\_demo, multi\_usecase\_demo

submission/ — five portal files: mp4 · README.pdf · Proposal.pdf · Pitch.pptx

Public GitHub — <https://github.com/Srujan0798/controlplane-ai>

Content laws (ARCHITECTURE §10) are enforced in CI: `pytest tests/test_content_laws.py`.